

F24-116-D-PestInsight

Project Team

Ali Maaz Cheema	21I-0636
Muhammad Bin Basit Masood	21I-2486
Syed Ali Tallian	21I-0599

Session 2021-2025

Supervised by

Dr. Labiba Fahad

Co-Supervised by

N/A



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2025

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	1
1.2.1	Understanding the IP102 Dataset	2
1.2.2	Challenges Involved	2
1.2.3	Creating Ensemble Model	3
1.2.3.1	Improved Generalizations	3
1.2.3.2	Handling Class Imbalance	3
1.2.3.3	Enhanced Robustness	4
1.2.3.4	Reduction of Error	4
1.3	Modules	4
1.3.1	Client Web App	4
1.3.2	IP-102 Dataset Preprocessing	5
1.3.2.1	Data Annotation	5
1.3.2.2	Data Augmentation	5
1.3.2.3	Data balancing:	5
1.3.3	Model Training and Fine Tuning	5
1.3.4	Back-End Operations	6
1.3.5	Database Operations	6
1.3.6	Testing	6
1.4	User Classes and Characteristics	7
2	Project Requirements	9
2.1	Use-case/Event Response Table/Storyboarding	9
2.2	Functional Requirements	15
2.2.1	User Registration	15
2.2.2	Image Upload for Pest Detection	16
2.2.3	Generate Diagnosis Report	16
2.2.4	View Diagnosis History	17
2.2.5	Chat with Chatbot	17
2.2.6	Display Pest Control Instructions	17
2.2.7	Image Enhancement	17

2.2.8	Assess Pest Intensity	18
2.2.9	User Profile Customization	18
2.3	Non-Functional Requirements	18
2.3.1	Reliability	18
2.3.2	Usability	19
2.3.3	Performance	19
2.3.4	Security	19
2.4	Domain Model	20
3	System Overview	21
3.1	Architectural Design	21
3.1.1	Architecture Components	22
3.1.1.1	Presentation Layer (Frontend - React)	22
3.1.1.2	Application Logic Layer (Backend - Django)	22
3.1.1.3	Business Logic Layer (Detection Module)	22
3.1.1.4	Data Management Layer (MySQL Database)	22
3.2	Design Models	23
3.2.1	Activity Diagram	23
3.2.2	System Sequence Diagram	24
3.2.3	Data Flow Diagram	29
3.2.4	State Transition Diagram	31
3.3	Data Design	31
3.3.1	Entity-Relationship Diagram	32
	References	37

List of Figures

1.1	IP102 Dataset	2
2.1	Use-case Diagram	9
2.2	Domain Model for PestInsight Web App	20
3.1	Multi-tiered MVC System Architecture	21
3.2	Activity Diagram	23
3.3	System Sequence Diagram for Registering User	24
3.4	System Sequence Diagram for Uploading Image	25
3.5	System Sequence Diagram for Generating a Report	26
3.6	System Sequence Diagram for Viewing Diagnosis History	27
3.7	System Sequence Diagram for Viewing Pest Management Tips	28
3.8	Data Flow Diagram Level 0	29
3.9	Data Flow Diagram Level 1	29
3.10	Data Flow Diagram Level 2	30
3.11	State Transition Diagram	31
3.12	ERD	32

List of Tables

Chapter 1

Introduction

1.1 Problem Statement

PestInsight, an AI-driven system aims to detect pests in major food crops namely wheat, rice and corn. It will rediscover crop protection by facilitating farmers in identifying pest infestations, eradicating them using sustainable farming techniques ultimately increasing crop yields. The global impact of harmful pests on major food crops like wheat, rice, corn etc. is devastating. According to FAO, every year pests account for significant financial losses totalling to about \$220 billion by destroying 40% of the global crop production. [1] Wheat, being one of the affected crops, faces losses of 30% [1] while rice faces losses between 10% to 50% [2] and corn faces losses up to 80%. [1] [3] Global food insecurity becomes apparent when such figures are highlighted. Moreover, the need for PestInsight becomes evident when traditional pest detection methods that are slow, exhaustive and labor-intensive, fail causing increased damage to crops, food shortages and hence economic losses. The excessive overuse of broad spectrum pesticides is also unable to solve the issue at hand further increasing the need for an effective pest detection system such as PestInsight.

1.2 Scope

The scope of PestInsight is focused on three major crops which are rice, wheat and corn. There are 14 different types of pests in rice and wheat and 12 different types of pests in corn. For the detection of pests on these three crops we will be using a subset of the IP-102 dataset [4] with 75,222 images in total. The number of pest images in the dataset corresponding to each crop are: Rice: 6322, Wheat: 2279, Corn: 2318.

1.2.1 Understanding the IP102 Dataset

Rice	Wheat	Corn
No. of Images = 6,322 No. of Classes = 14 1. Rice Leaf Roller 2. Rice Leaf Caterpillar 3. Paddy Stem Maggot 4. Asiatic Rice Borer 5. Yellow Rice Borer 6. Rice Gall Midge 7. Rice Stemfly 8. Brown Plant Hopper 9. White Backed Plant Hopper 10. Small Brown Plant Hopper 11. Rice Water Weevil 12. Rice Leafhopper 13. Grain Spreader Thrips 14. Rice Shell Pest	No. of Images = 2,279 No. of Classes = 14 1. English Grain Aphid 2. Green Bug 3. Bird Cherry-Oat Aphid 4. Wheat Blossom Midge 5. Penthaleus Major 6. Longlegged Spider Mite 7. Wheat Phloeothrips 8. Wheat Sawfly 9. Cerodonta Denticornis 10. Beet Fly 11. Flea Beetle 12. Cabbage Army Worm 13. Beet Army Worm 14. Meadow Moth	No. of Images = 2,318 No. of Classes = 12 1. Corn Borer 2. Army Worm 3. Aphids 4. Wireworm 5. Black Cutworm 6. Large Cutworm 7. Yellow Cutworm 8. Flea Beetle 9. Cabbage Army Worm 10. Beet Army Worm 11. Meadow Moth 12. Mole Cricket

Figure 1.1: IP102 Dataset

We will be training and fine tuning our ensemble model using the Ultralytics YOLOv8 and Faster R-CNN for real time image analysis to provide us with accurate results. The goal of the model is to detect pests and count them. The uploaded images at run time will be enhanced to improve the accuracy of our model. For training our model the images will be annotated using RoboFlow. To make it easy for the user our platform will be a web app with a friendly user interface which we will develop using Django, React and MYSQL. PestInsight will also offer severity assessments, pest management tips and provide the user with the option to generate detailed reports about the results. Additionally, a built-in chatbot will be integrated to assist and guide the user.

1.2.2 Challenges Involved

Our scope has a few challenges that we have to overcome:

1. Annotation Format
2. Inter-class Similarities in Dataset
3. Intra-class Variability within Dataset

4. Lifecycle Stages of Pests
5. Distinguishing Similar Pests
6. Class Imbalance
7. Model Performance Optimization
8. Limitations of Existing Solutions

Firstly, the dataset is not annotated in the format required by our models so we will need to convert the images accordingly. The dataset has inter-class similarities in which different pests look very similar and intra-class similarities in which pests species are same but they look different due to several factors which we will overcome by balancing the dataset to ensure that our model gets enough examples of each. Additionally, there is also a class imbalance which can will be dealt by undersampling and oversampling. Finally, we will overcome the limitations of existing apps by incorporating features such as pest counting, detecting all the stages of pest lifecycle and will provide support to upload multiple images simultaneously.

1.2.3 Creating Ensemble Model

The improvement of object detection will be achieved by using an ensemble approach that integrates YOLOv8 and Faster R-CNN models. Furthermore it is useful in complex data such as the IP102 dataset with several pest classes and potential class imbalance issues. The following explains why ensemble modeling is useful in this situation:

1.2.3.1 Improved Generalizations

Different models have different strengths and weaknesses. While Faster R-CNN focuses on accuracy, YOLOv8 is said to be highly efficient in terms of speed and performance in real-time applications. Combining these models allows us to take use of their features, which improves overall performance on a heterogeneous dataset such as IP102.

1.2.3.2 Handling Class Imbalance

The IP102 dataset shows imbalances both intra- and interclass, or between and within a class, indicating that certain pest classes are more common than others. By combining predictions from several models, ensemble approaches might help lessen these problems by balancing the biases of individual models toward particular classes. This can improve the model's capacity to identify less common classes that a single model might miss.

1.2.3.3 Enhanced Robustness

In general, ensemble models are less sensitive to the noise present in a dataset and variations in such a set. This is important especially for pest recognition because distinctly different backgrounds, environmental conditions and image contrast and resolution alter the appearance of a pest.

1.2.3.4 Reduction of Error

The ensemble model can reduce the errors created by different models by combining the outputs of several models in a single one. In total, this can lead to better means of prediction, especially in such a complex case as pest identification, whereas false positives and false negatives can have a very negative outcome.

The IP102 dataset presents a number of issues, mostly related to inter-class and intra-class imbalances. These challenges can in fact be ably managed if YOLOv8 is implemented in conjunction with Faster R-CNN in an ensemble manner. These models' capabilities of learning can be quite useful in determining and classifying numerous pests accurately.

1.3 Modules

1.3.1 Client Web App

Module featuring front end where the user will interact with the web application. The React front-end will ensure a smooth, dynamic easy to use interface, also enabling users to perform their desired tasks by navigating through different options.

1. Sign Up and Log In
2. Upload Image for Pest Detection
3. Generate Diagnosis Report
4. History of Diagnosis
5. Chat with ChatBot
6. Pest Management Tips
7. Severity Assessment
8. Customize Profile
9. Image Quality Enhancement

1.3.2 IP-102 Dataset Preprocessing

Roboflow would be used for the preprocessing of IP-102 dataset. Before the models are trained, some preprocessing steps are required to get the best performance. The major preprocessing techniques are:

1.3.2.1 Data Annotation

Images from the dataset would be annotated by manually drawing bounding boxes around objects required for detection and classifying them into classes. Corresponding testing, training and validation data would be used for model training.

1.3.2.2 Data Augmentation

Rotation: Facilitates object detection by helping models learn from different angles. **Flipping:** Facilitates model's robustness against variations in object detection by flipping images vertically and horizontally. **Scaling and resizing:** Facilitates model in better detecting objects by randomly resizing images **Color Jitter:** Facilitates models to become invariant to lighting conditions by adjusting brightness, saturation and contrast of images.

1.3.2.3 Data balancing:

Since the dataset has a class imbalance, oversampling and undersampling will be considered to ensure the models are trained on a balanced dataset.

We will experiment with different combinations of these preprocessing steps to increase the quality of the training data and to find models that perform the best.

1. Dataset Annotation and Augmentation for Rice (14 Classes)
2. Dataset Annotation and Augmentation for Wheat (14 Classes)
3. Dataset Annotation and Augmentation for Corn (12 Classes)

1.3.3 Model Training and Fine Tuning

This module includes individually training and fine tuning models on the processed datasets of wheat, rice and corn. And then creating, training and fine tuning a final ensemble model incorporating YOLOv8 and Faster R-CNN.

1. Training Ultralytics YOLOv8
2. Training Faster R-CNN
3. Training final ensemble model

1.3.4 Back-End Operations

The Django python back-end will integrate the front-end user interface with the final ensemble model, allowing users to communicate with the system. It processes users actions to provide real time results through the trained model. It also manages the user data efficiently by communicating with the database.

1. Secure User Authentication
2. Integrating front-end with trained ensemble model
3. API calling for communication between components
4. Database communication

1.3.5 Database Operations

The database will be used to store data to ensure users authenticity and keep their data secure and easily available.

1. User personal data
2. User uploaded image
3. User report diagnosis

1.3.6 Testing

Testing will ensure the software is running as intended, performance and results are optimal, different components and models are interacting as expected.

1. Unit Testing
2. Performance Testing
3. Integration Testing

1.4 User Classes and Characteristics

1. Introduction

User class	Description
Farmer	A farmer is the primary user of the PestInsight app. They will upload images of their crops to receive pest identification, detailed diagnosis reports, and tailored pest management tips. The app's interface is designed to be intuitive and user-friendly, requiring only basic technical skills, ensuring ease of use for farmers. The farmer will interact with the app primarily during farming sessions or whenever pest detection is needed. PestInsight aims to deliver fast, accurate, and actionable results to assist in timely pest management and crop protection.

Example: User Classes and Characteristics

Chapter 2

Project Requirements

2.1 Use-case/Event Response Table/Storyboarding

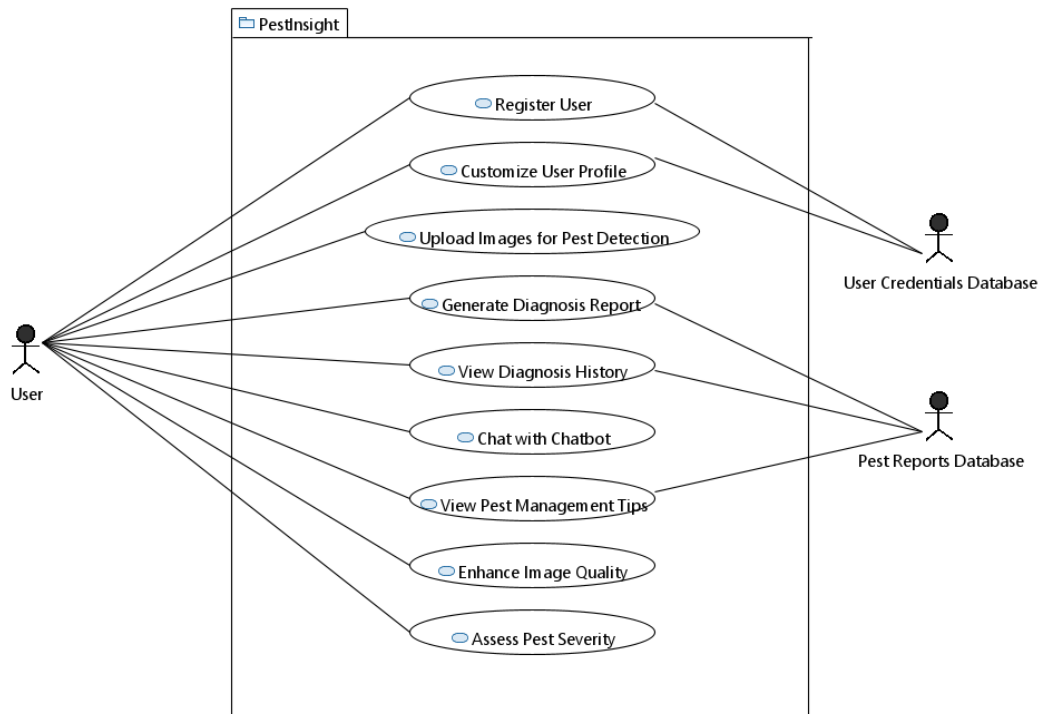


Figure 2.1: Use-case Diagram

Use Case	Register User
Scope	PestInsight Web Application
Level	User goal
Primary Actors	User
Stakeholders and Interests	User: Wants to register and get secure access to the web app. System Administrator: Wants to ensure only authorized users access the web app.
Preconditions	User is not registered.
Success guarantees (postconditions)	User is registered, authenticated and has access to the web app.
Main Success Scenario	1. User navigates to the Sign Up page. 2. User enters a unique username and password. 3. System prompts the user to enter personal information such as name and contact information. 4. User enters the information. 5. System validates the information entered and registers the user.
Extensions	2a. Username is already taken. 1. Prompt the user to choose another username. 2b. Password is not strong enough. 1. Prompt the user to choose a stronger password. 4. Personal information is invalid or incomplete. 1. Inform the user about the errors and prompt to enter correct information. 5. System is unable to validate the user. 1. Inform the user about the errors and prompt them to contact the system administrator.
Special Requirements	Secure password handling

Use Case	Upload Images for Pest detection
Scope	PestInsight Web Application
Level	User goal
Primary Actors	User
Stakeholders and Interests	User: Wants to upload images for accurate pest detection. Farmers: Need precise pest identification for crop health.
Preconditions	User is logged in.
Success guarantees (postconditions)	The system processes the uploaded images and provides pest detection results.
Main Success Scenario	1. User selects the "Upload Images" option. 2. User uploads images of crops. 3. The system enhances the images. 4. The system applies a pest detection ensemble model. 5. The system returns detected pests with counts to the user.
Extensions	2. Image upload fails due to unsupported format. The system prompts for re-upload.
Special Requirements	Image size and format (PNG, JPEG) should be supported.
	Processing time should be under 15 seconds for a positive user experience.

Use Case	Generate Diagnosis Report
Scope	PestInsight Web Application
Level	User goal
Primary Actors	User
Stakeholders and Interests	User: Wants detail pest identification and detection results reported Farmers: Want printable report for further actions.
Preconditions	Pest detection has been conducted.
Success guarantees (postconditions)	A downloadable report is generated successfully.
Main Success Scenario	1. User selects the "Generate Diagnostic Report" option. 2. System compiles results into a report. 3. System generates a corresponding PDF. 4. User downloads the PDF report.
Extensions	2a. If no pests are detected, the system informs the user that no report can be generated.
Special Requirements	Report should be generated in less than 10 seconds.

Use Case	View Diagnosis History
Scope	PestInsight Web Application
Level	User goal
Primary Actors	User
Stakeholders and Interests	User: Wants to view the previous diagnosis results available. System Administrator: Wants to ensure data is securely stored and retrieved.
Preconditions	User is logged in and has previously generated pest detection reports.
Success guarantees (postconditions)	User can view the list of previous diagnosis and access detailed reports.
Main Success Scenario	1. User navigates to the “History” section. 2. System retrieves all the previous diagnoses generated from the user’s account. 3. User selects a previous diagnosis and views a detailed report.
Extensions	2a. Prompt the user if no previous diagnosis report is available.
Special Requirements	Data should be retrieved and showed in less than 5 seconds.

Use Case	Chat with Chatbot
Scope	PestInsight Web Application
Level	User goal
Primary Actors	User
Stakeholders and Interests	User: Wants timely assistance and guidance. System Administrator: Wants to ensure the chatbot provides correct and useful information.
Preconditions	User is logged in and the chatbot functionality is available.
Success guarantees (postconditions)	Relevant assistance is provided to the user in response to his query.
Main Success Scenario	1. User initiates a chat with the chatbot on the web application. 2. User enters a query. 3. Chatbot replies with assistance and relevant information.
Extensions	3a. Chatbot asks for clarification and more details if it is unable to comprehend the query. 3b. Chatbot prompts the user to ask for human support if it fails to provide assistance.
Special Requirements	Chatbot response time should be less than 5 seconds.

Use Case	View Pest Management Tips
Scope	PestInsight Web Application
Level	User Goal
Primary Actors	User
Stakeholders and Interests	User: Wants tips on how the identified pests can be dealt with. Farmers: Want effective pest management tips.
Preconditions	Pests have been detected and identified in the images uploaded by the user.
Success guarantees (postconditions)	Pest-specific management tips are given to the user.
Main Success Scenario	1. User views the identified pests. 2. User selects a pest from the results displayed. 3. The system displays pest-specific management tips to the user.
Extensions	3a. Inform the user if specific pest management tips are unavailable.
Special Requirements	The tips should be obtained from reliable agricultural sources.

Use Case	Enhance Image Quality
Scope	PestInsight Web Application
Level	Subfunction
Primary Actors	User
Stakeholders and Interests	User: Wants greater changes of accurate pest detection through enhancing image quality.
Preconditions	The user has uploaded images for pest detection.
Success guarantees (postconditions)	The user's uploaded image's quality is enhanced successfully.
Main Success Scenario	1. User uploads an image for pest identification. 2. The system enhances the image quality. 3. The system uses the enhanced image for pest identification.
Extensions	2a. The system informs the user if the image quality can not be further enhanced.
Special Requirements	The enhancement process should take less than 10 seconds.

Use Case	Assess Pest Severity
Scope	PestInsight Web Application
Level	User goal
Primary Actors	User
Stakeholders and Interests	User: Wants to know the severity of the pest infestation to take timely and appropriate countermeasures. Farmers: Wants to know the severity of the pest infestation to take timely and appropriate countermeasures.
Preconditions	Pest detection is completed.
Success guarantees (postconditions)	The system provides a complete severity analysis of the pest infestation including pest types, their counts and potential impact.
Main Success Scenario	1. User requests the severity assessment. 2. The system calculates the severity of the infestation based on the pest types and their counts. 3. The system displays the severity level based on the calculations performed. The levels are low, moderate, and severe.
Extensions	3a. The system informs the user that no severity assessment can be generated if there are no pests detected.
Special Requirements	The system should provide the severity assessment within 10 seconds. The system should calculate the severity assessment based on some predefined criteria.

Use Case	Customize User Profile
Scope	PestInsight Web Application
Level	User goal
Primary Actors	User
Stakeholders and Interests	User: Wants to customize their profile to include personal preferences. System Administrator: Wants user retention through personalized features.
Preconditions	The user is logged in.
Success guarantees (postconditions)	The user's profile is successfully updated after the customization.
Main Success Scenario	1. User navigates to the profile customization section. 2. User edits their personal information. 3. User submits the updated information. 4. The system validates the inputs and updates changes in the user database. 5. The system prompts the user that the information has successfully been updated.
Extensions	2. The system prompts the users to fill in any required field if it is missing. 4. The system asks the user to rectify the input if any validation fails. 5. Incase of system error, the system notifies the user and asks to try again later.
Special Requirements	Profile information should be securely stored and retrieved.
	All changes should be reflected in real time.

2.2 Functional Requirements

2.2.1 User Registration

Following are the requirements for this module:

1. The system shall allow users to register with a unique username, password, and personal information, for example name and contact details.
2. The system shall validate the personal information provided by the user during registration.

3. In the event that the username is already taken, the system will ask the user to re-enter the details again.
4. The system shall require password complexity rules to be followed by the user and will ask the user to choose a stronger password if necessary.

2.2.2 Image Upload for Pest Detection

Following are the requirements for this module:

1. The system shall introduce logged-in users to a situation where they can upload images of crops that are suspected to have pests.
2. The system shall make the uploaded image more suitable to be used for the recognition of pests so as to improve the mentioned problem.
3. The system shall employ pest identification models (YOLOv8, R-CNN) to warrant pest classification as well as to provide further information such as the titles of specific pests and counts.
4. The system shall be capable of reading image formats like PNG and JPEG.
5. The system shall keep the user informed if the image upload fails because of unsupported formats or large file sizes.

2.2.3 Generate Diagnosis Report

Following are the requirements for this module:

1. The system shall allow users to generate a detailed diagnostic report based on the pest detection results.
2. The system shall compile pest types, counts, and severity levels into a report.
3. The system shall notify users if no pests are detected and a report cannot be generated.
4. The system shall generate reports within 10 seconds.

2.2.4 View Diagnosis History

Following are the requirements for this module:

1. The system shall allow users to view previously generated diagnosis reports from their account.
2. The system shall retrieve and display all previous diagnosis results in the "History" section within 5 seconds.
3. The system shall notify users if no past reports are available.

2.2.5 Chat with Chatbot

Following are the requirements for this module:

1. The system shall include a chatbot feature to assist users with queries related to pest detection and platform usage.
2. The chatbot shall respond to user queries within 5 seconds.
3. The chatbot shall ask users for clarification if it does not understand the query.
4. The chatbot shall direct users to human support if it fails to resolve the user's issue.

2.2.6 Display Pest Control Instructions

Following are the requirements for this module:

1. The system shall display pest control instructions after the pests are identified from the uploaded images.
2. The system shall retrieve pest control instructions from database.
3. The system shall alert the users if the pest-specific control instructions are not available.

2.2.7 Image Enhancement

Following are the requirements for this module:

1. The system shall upload photos by improving the quality to analyze the pests accurately.

2. The system shall notify the users that the system will not be able to enhance the quality of the image.
3. The time taken for the image enhancement process shall not exceed 10 seconds.

2.2.8 Assess Pest Intensity

Following are the requirements for this module:

1. The system shall grade the intensity of the infestation based on the types of pests and their count identified.
2. The system shall grade the intensity of the infestation as low, moderate, or severe within 10 seconds following the end of the process of detection of the pest.
3. The system shall notify the user that no risk score can be generated because of the absence of the pest.

2.2.9 User Profile Customization

Following are the requirements for this module:

1. The system shall have the feature of enabling users to update and change their personal information in the profile.
2. The system shall validate the updated profile for the presence of errors before saving it.
3. The system shall notify the user when some or all the mandatory fields are absent or invalid.
4. System shall update safely all profile information and keep them reflected in real time.

2.3 Non-Functional Requirements

2.3.1 Reliability

Use-1: Software failure: The system shall log error messages with details for further analysis, human readable error messages on what is wrong maybe even on what the user will do next.

Use-2: Error detection mechanisms will be provided within the application to be used in monitoring system health state and reporting possible failures to administrators.

2.3.2 Usability

USE-1: The PestInsight application shall enable users to obtain access to past results of pest detection through one interaction from the dashboard.

USE-2: The system shall offer an intuitive interface with clear and labeled buttons, by which instructions shall guide users on how to navigate through the processes for pest detection and reporting.

USE-3 : In the application, it shall ensure that there are no more than three single steps applied for uploading an image, in order to realize pest detection.

USE-4: By the system, accessibility compliance shall be ensured whereby people with disabilities shall navigate as well as use all functionalities effectively.

2.3.3 Performance

PER-1: The PestInsight application shall upload images and respond with pest detection results within 10 seconds, with an interactive response.

PER-2: 95% of all reports generated by the system shall be downloadable within 10 seconds of a user's request for a report.

PER-3: The system shall maintain a fast data retrieval time to access priorly made reports.

2.3.4 Security

SEC-1: The PestInsight application will implement role-based access control to prevent the exposure of sensitive data and functionality to unauthorized users.

SEC-2: The system shall undergo regular security audits and vulnerability assessments at least every 6 months to identify and address potential security risks.

2.4 Domain Model

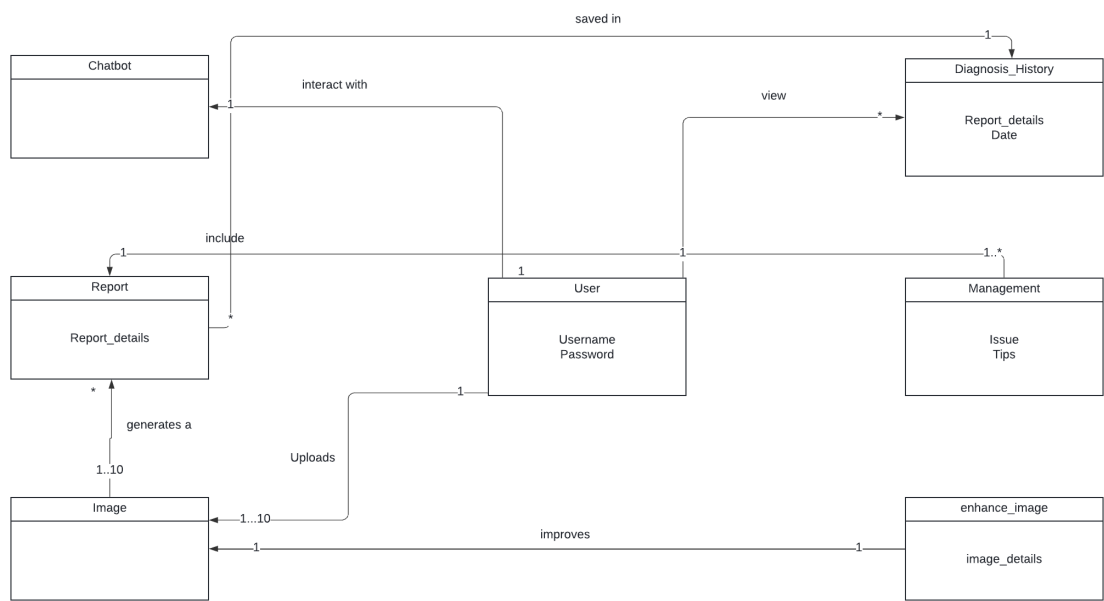


Figure 2.2: Domain Model for PestInsight Web App

Chapter 3

System Overview

PestInsight is a web-based interface dedicated to the recognition of pests in rice, wheat and corn crops based on real-time deep learning models (YOLOv8, Faster R-CNN). It generates reports, estimates severity and provides advice on pest management. The system consists of a React frontend, Django backend and MySQL for database management. PestInsight tackles problems such as class imbalance, and similarities between pests by firstly preparing the dataset and then enhancing image quality for better recognition.

3.1 Architectural Design

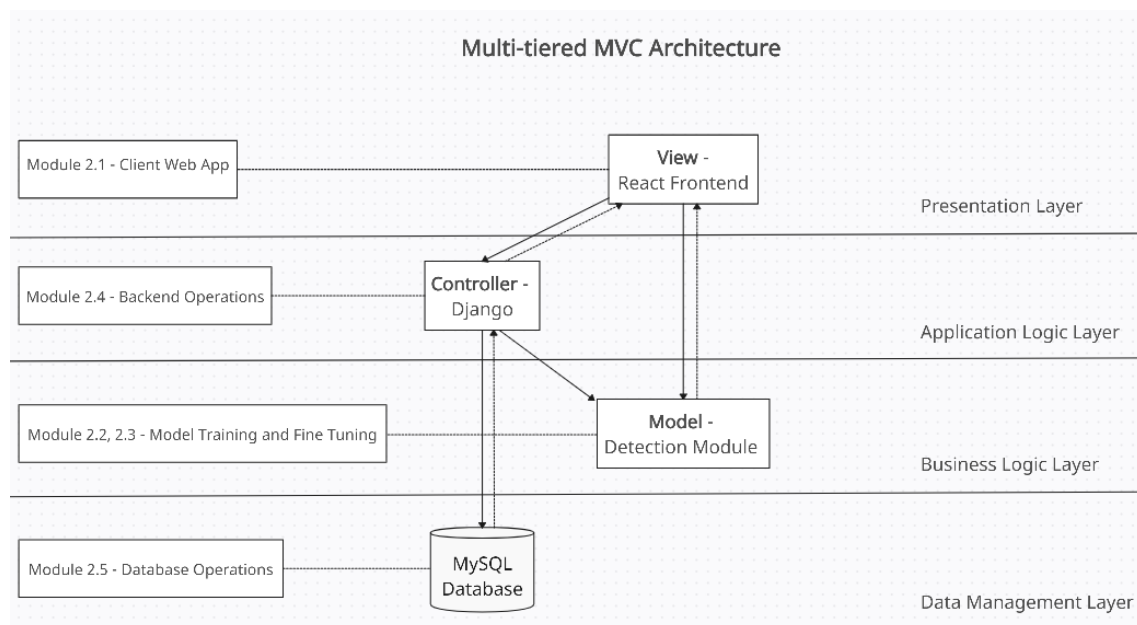


Figure 3.1: Multi-tiered MVC System Architecture

3.1.1 Architecture Components

PestInsight uses a multi-tiered layered architecture with a Model View Controller (MVC) pattern that separates concerns promising scalability, testing and maintainability.

3.1.1.1 Presentation Layer (Frontend - React)

User Interaction: The user's role begins with logging on to the web page, uploading images, viewing results and severity of pest detection. Chatbot works for help and guidance.

3.1.1.2 Application Logic Layer (Backend - Django)

Controller: The controller is in charge of routing, managing requests, activating models, and performing database interactions.

Image Handling: Requests concerning pest detection are directed to YOLOv8 and Faster R-CNN models.

3.1.1.3 Business Logic Layer (Detection Module)

Pest Detection: The pest detection model works on the images provided and gives an analysis of the results.

Image Processing: This improves the quality of any given image in a bid to increase the accuracy level.

3.1.1.4 Data Management Layer (MySQL Database)

Storage: Manages the user's personal data, saves the results after detection and manages the generation of reports.

Retrieval: Also, historical data is made available to users and assists in tracking previous diagnoses.

3.2 Design Models

3.2.1 Activity Diagram

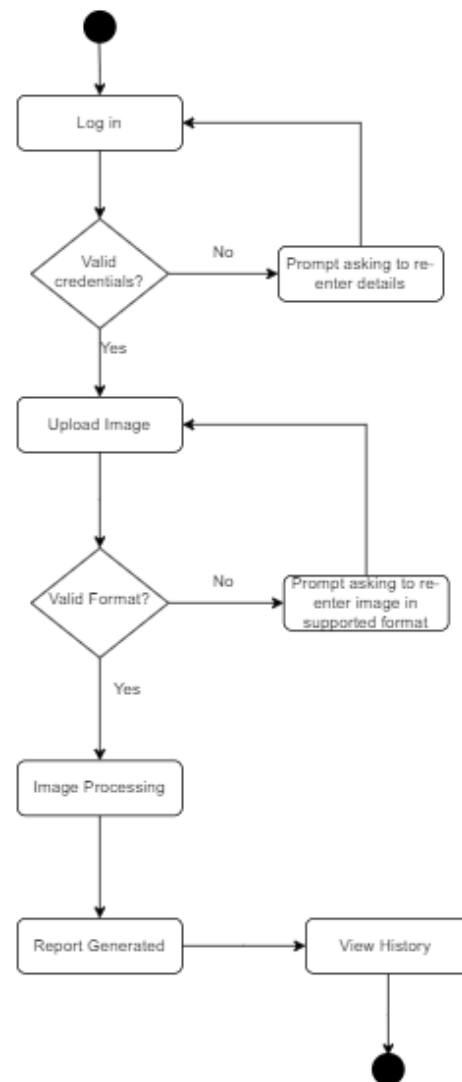


Figure 3.2: Activity Diagram

3.2.2 System Sequence Diagram

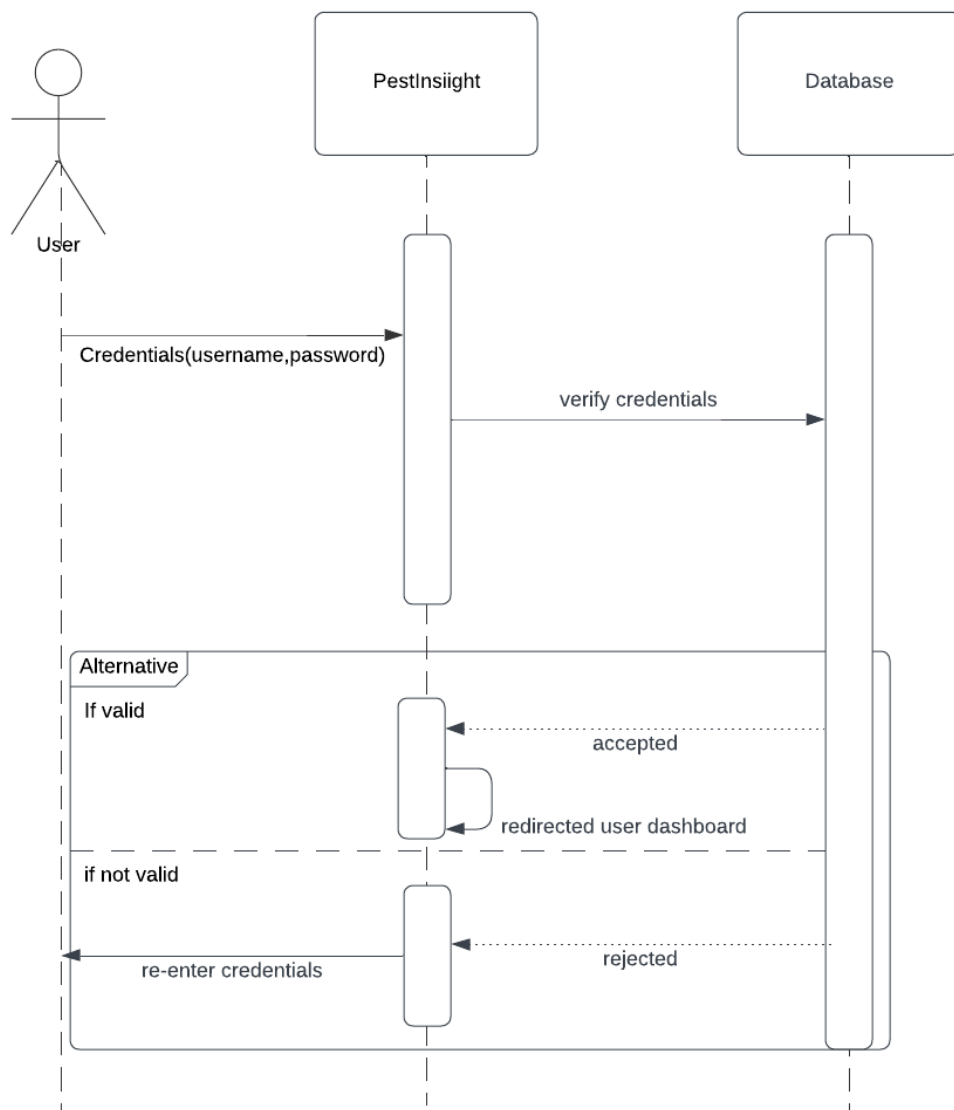


Figure 3.3: System Sequence Diagram for Registering User

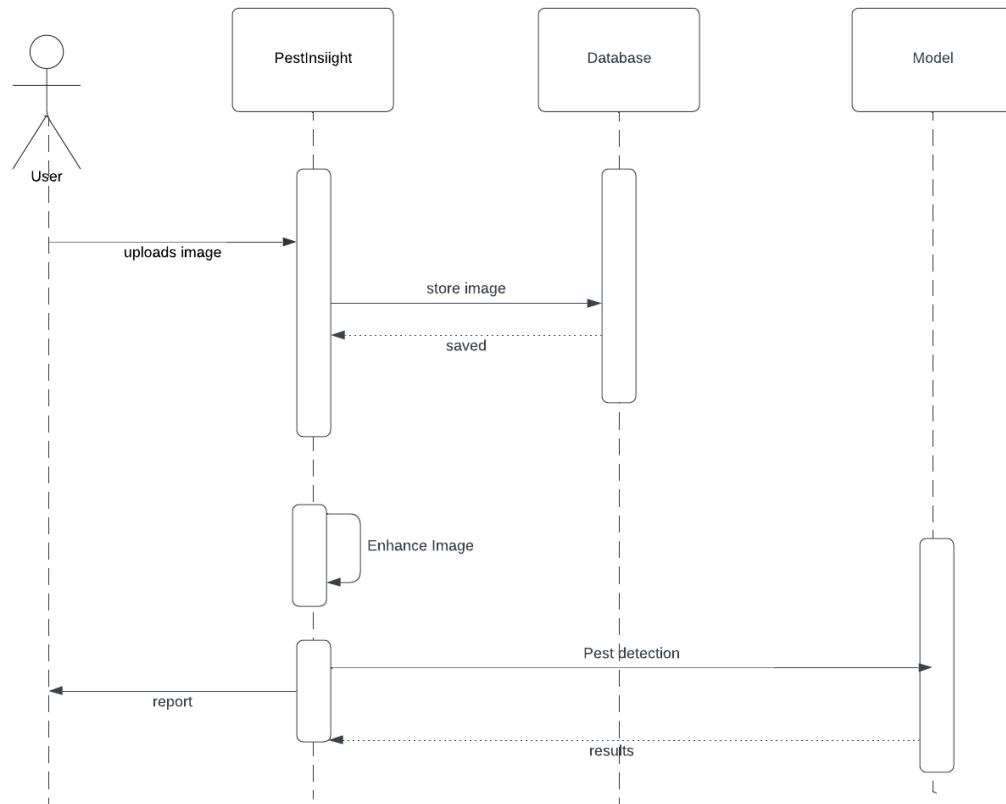


Figure 3.4: System Sequence Diagram for Uploading Image

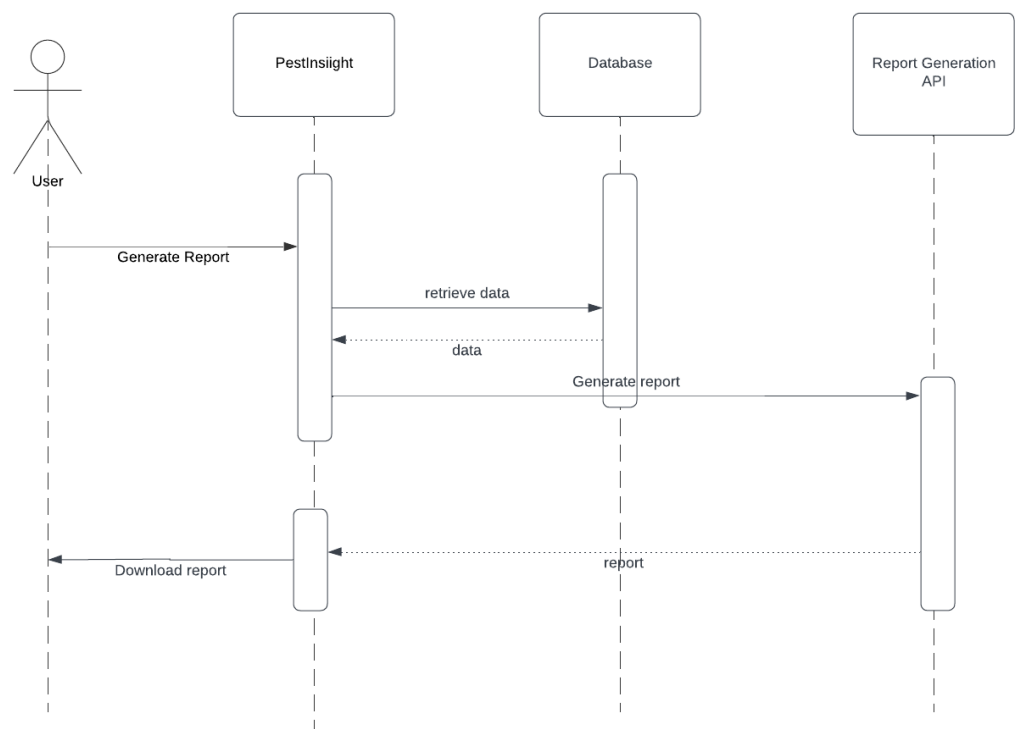


Figure 3.5: System Sequence Diagram for Generating a Report

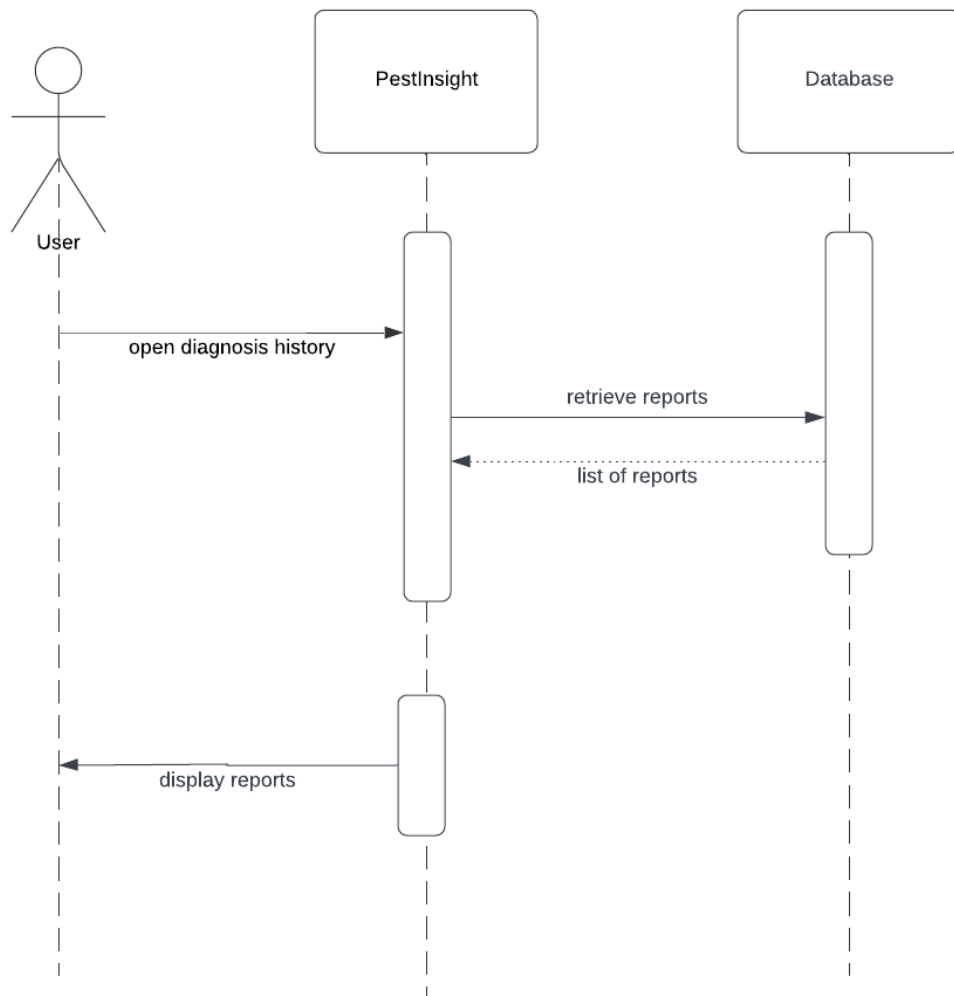


Figure 3.6: System Sequence Diagram for Viewing Diagnosis History

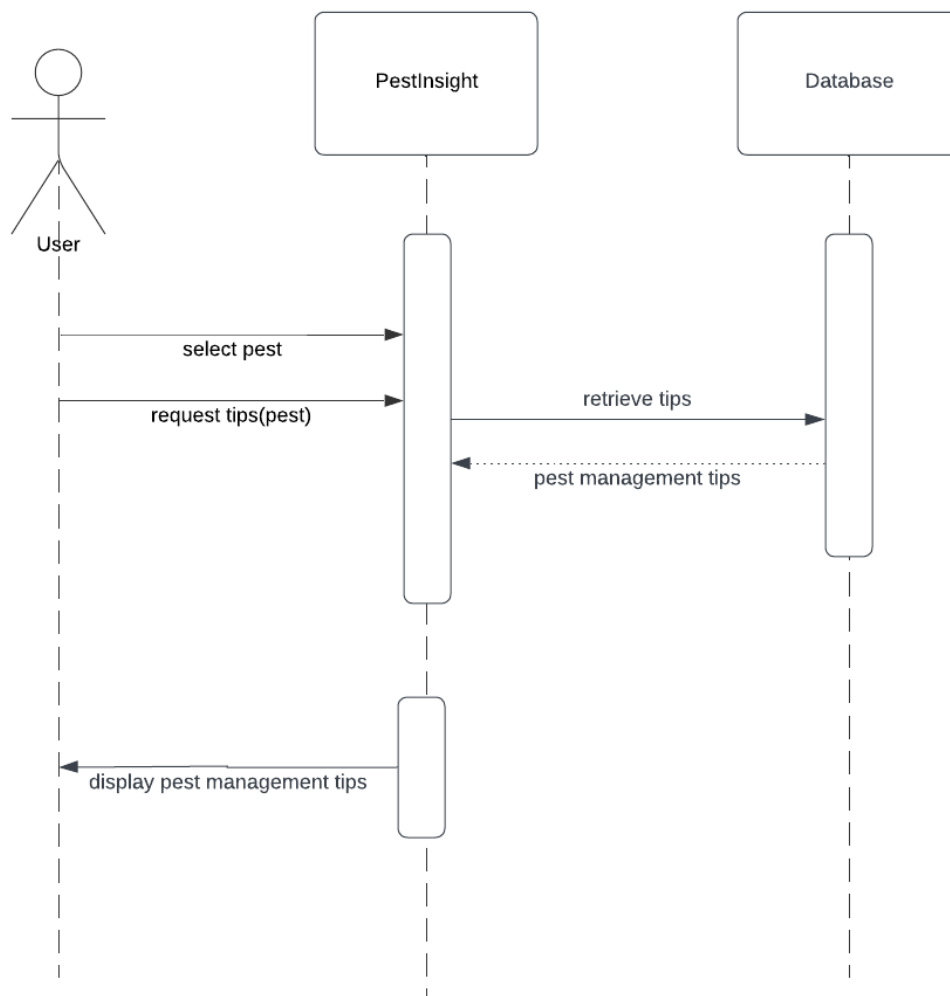


Figure 3.7: System Sequence Diagram for Viewing Pest Management Tips

3.2.3 Data Flow Diagram



Figure 3.8: Data Flow Diagram Level 0

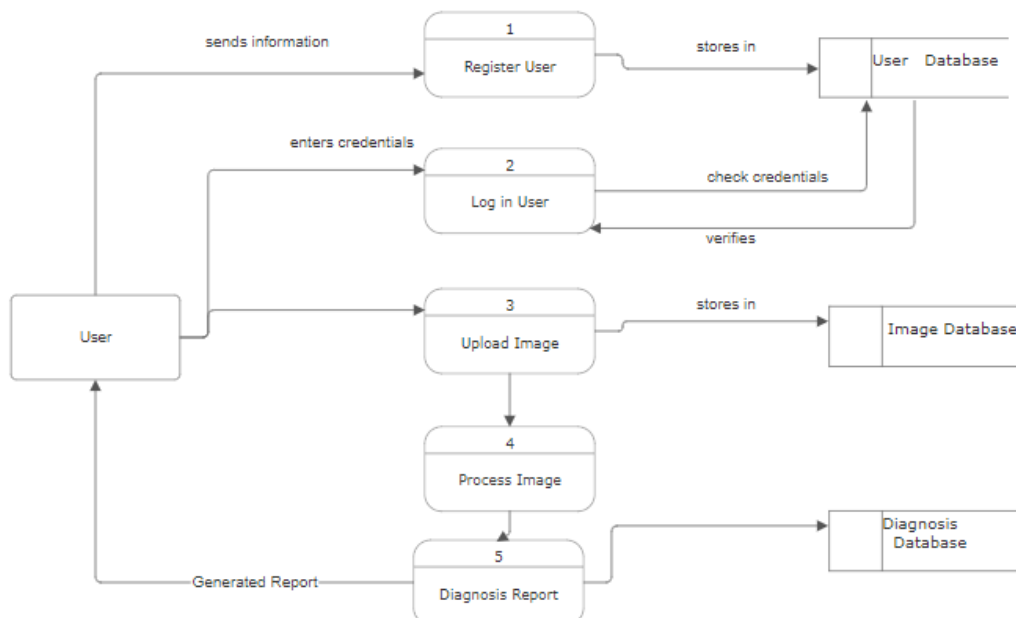


Figure 3.9: Data Flow Diagram Level 1

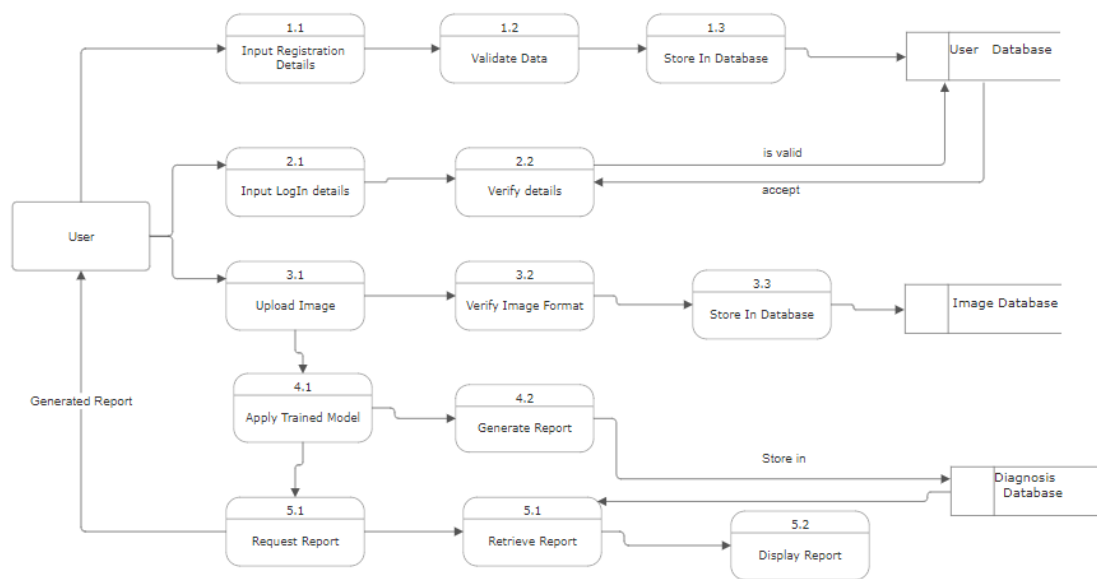


Figure 3.10: Data Flow Diagram Level 2

3.2.4 State Transition Diagram

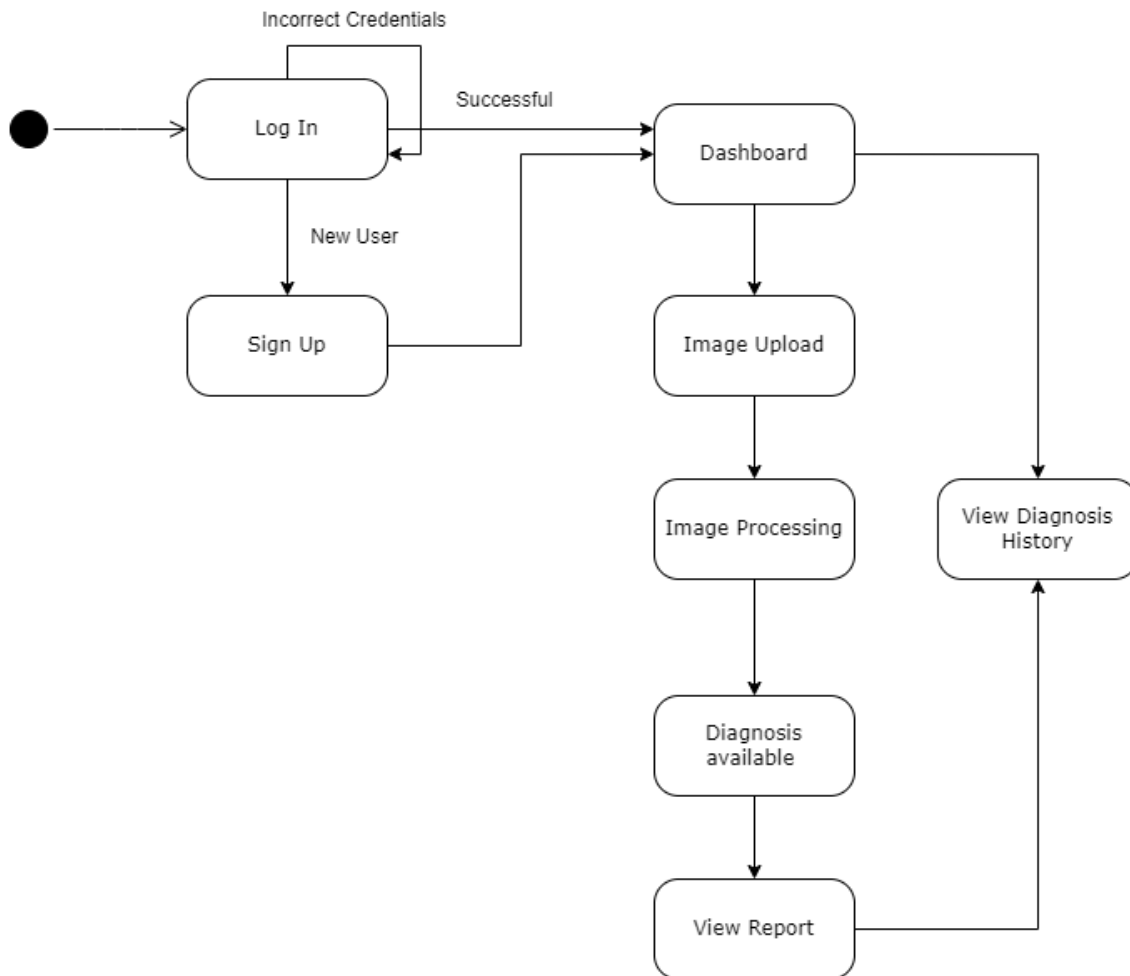


Figure 3.11: State Transition Diagram

3.3 Data Design

The PestInsight system operates within a MySQL database and encompasses various components, including users, pest detection images, pest data, reports, and system settings. These components are organized into clearly defined relational tables that enhance the system's functionality, allowing for effective data storage, retrieval, and processing. Each component is represented by specific tables in the MySQL database, featuring fields that capture details such as user information, metadata for uploaded images, pest detection outcomes, and generated reports. Foreign key relationships among the tables enable smooth interactions between components, ensuring accurate connections between users, images, and pest reports. This relational framework in MySQL promotes efficient query

execution and data management, facilitating seamless communication between the system's backend and user interface, which ultimately provides a responsive and scalable application experience.

3.3.1 Entity-Relationship Diagram

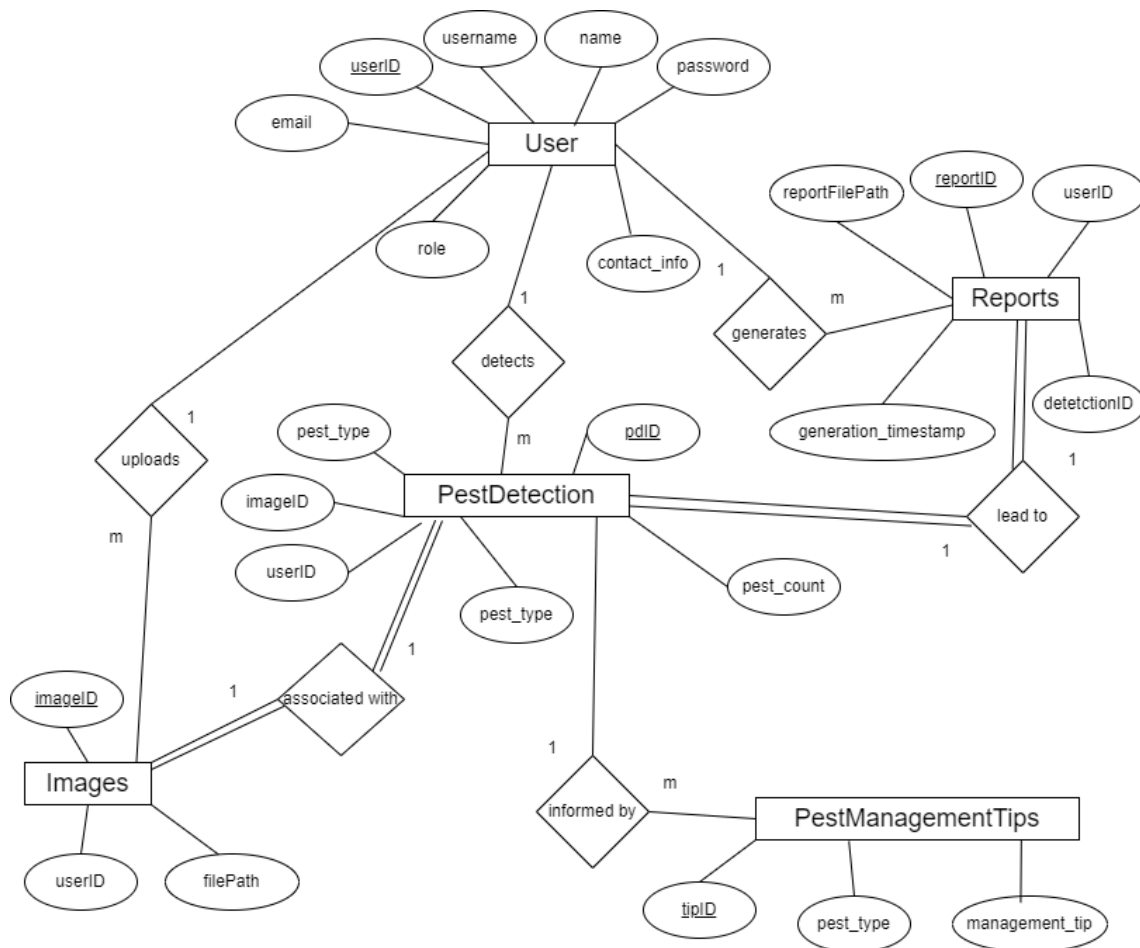


Figure 3.12: ERD

MySQL Database Structure for PestInsight

1. User Data

- **Table:** users
- **Fields:**
 - user_id (Primary Key, AutoIncrement)

- username (Unique)
- password
- email (Unique)
- name
- contact_info
- role (e.g., admin, user)

- **Processing:**

- User records are created during registration and stored with hashed passwords.
- The role field determines user access levels (e.g., admin or regular user).
- Users can update their profiles, and changes are stored in the same table.

2. Image Data (Uploaded Crop Images)

- **Table:** images

- **Fields:**

- image_id (Primary Key, AutoIncrement)
- user_id (Foreign Key referencing users)
- file_path (Location of the image in file storage)

- **Processing:**

- Image is stored along with a reference to the user who uploaded the image.
- Each uploaded image is linked to a pest detection record in the pest_detections table.

3. Pest Detection Data

- **Table:** pest_detections

- **Fields:**

- detection_id (Primary Key, AutoIncrement)
- image_id (Foreign Key referencing images)
- user_id (Foreign Key referencing users)
- pest_type (List of detected pests)
- pest_count (Number of pests detected)
- severity_level (Low, Moderate, High)

- **Processing:**

- When an image is processed, pest detection results are stored in this table.

- The pest_type and pest_count fields are populated based on the AI model's output.
- severity_level is calculated and stored based on the pest count.

4. Diagnosis Report Data

- **Table:** reports
- **Fields:**
 - report_id (Primary Key, AutoIncrement)
 - user_id (Foreign Key referencing users)
 - detection_id (Foreign Key referencing pest_detections)
 - report_file_path (Location of the PDF report)
 - generation_timestamp (Timestamp)
- **Processing:**
 - After pest detection, a report is generated and stored as a downloadable PDF.
 - The report is linked to the pest detection and user data.

5. Pest Management Tips Data

- **Table:** pest_management_tips
- **Fields:**
 - tip_id (Primary Key, AutoIncrement)
 - pest_type
 - management_tip
- **Processing:**
 - Pest-specific management tips are retrieved and stored in this table, referenced when pests are detected.

Summary of MySQL Database Structure

- **Primary Tables:** users, images, pest_detections, reports, pest_management_tips
- **Foreign Key Relationships:**
 - user_id links many tables to the users table.
 - image_id links images to pest detection records.
 - detection_id links reports to pest detections.

This MySQL structure ensures that all major entities in the PestInsight system are efficiently stored, processed, and organized, allowing for smooth retrieval and processing of pest detection data, user information, and reports.

Bibliography

- [1] Faostat. *Food and agriculture data*.
- [2] Global pest management in rice. *IRRI (International Rice Research Institute)*.
- [3] Pests and plant diseases: A global perspective. *CABI (Centre for Agriculture and Bioscience International)*.
- [4] Xiaoping Wu, Chi Zhan, Yukun Lai, Ming-Ming Cheng, and Jufeng Yang. Ip102: A large-scale benchmark dataset for insect pest recognition. In *IEEE CVPR*, pages 8787–8796, 2019.